

# Elementos HTML

Un elemento HTML está formado por:

- Una etiqueta de apertura.
- Cero o más atributos.
- Opcionalmente, un texto, encerrado entre las etiquetas de apertura y cierre. No todas las etiquetas pueden encerrar texto.
- Una etiqueta de cierre. Para algunos elementos no hay etiqueta de cierre o es opcional.

Según el modo en que ocupan el espacio disponible en la página, los elementos pueden ser de dos tipos:

- Elementos en línea (*inline*) o lineales. Solo ocupan el espacio necesario para mostrar sus contenidos. Su contenido puede ser texto u otros elementos *inline*.
- Elementos de bloque (*block*). Los elementos de bloque siempre empiezan en una nueva línea y ocupan todo el espacio disponible hasta el final de la línea, aunque sus contenidos no ocupen todo el ancho. Su contenido puede ser texto, elementos *inline* u otros elementos de bloque.

## Ejemplo #

El siguiente ejemplo muestra la diferencia entre ambos comportamientos:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>
      Ejemplo de la diferencia entre los elementos inline y los elementos block
    </title>
  </head>
  <body>
    <h1>Los encabezados son elementos block.</h1>
    <p>Y los párrafos también.</p>
    <a href="#">Los enlaces son elementos inline</a>
    <p>
      Incluso si esta definido dentro de un párrafo,
      <strong>un texto resaltado</strong> sigue siendo un elemento inline.
    </p>
```

```
</body>
</html>
```

Un navegador web lo mostraría así:

## Los encabezados son elementos block.

Y los párrafos también.

[Los enlaces son elementos inline](#)

Incluso si está definido dentro de un párrafo, **un texto resaltado** sigue siendo un elemento inline.

Pruébalo en el navegador pulsando aquí

## Secciones #

### Elemento `<div>` #

El elemento `<div>` se usa para marcar divisiones y agrupar otros elementos en secciones, tanto para organizar el contenido como para posicionarlo mediante hojas de estilo CSS.

```
<div>
  <div>Sección 1</div>
  <div>Sección 2</div>
</div>
```



## Etiquetas semánticas #

En HTML5 aparecieron varias etiquetas semánticas para estructurar el contenido de la página y, por tanto, solo se debería usar `<div>` cuando **no haya una etiqueta más apropiada**.

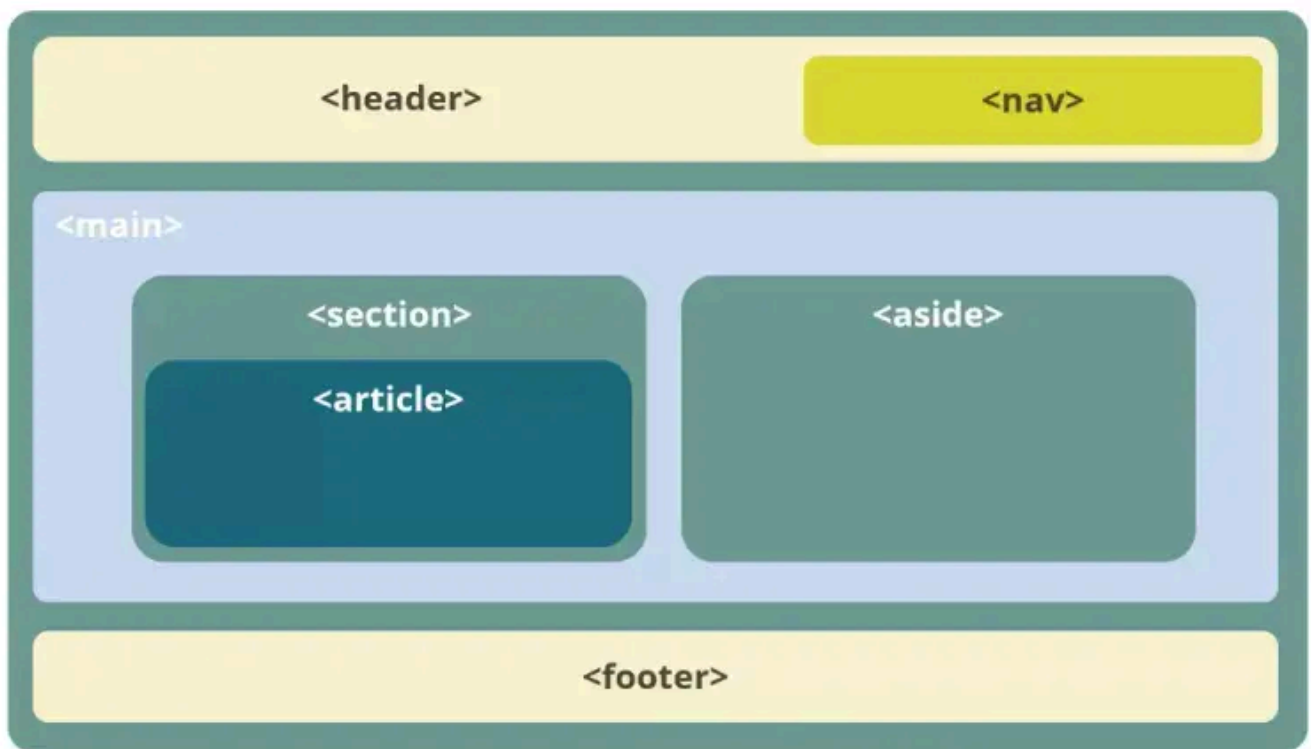
Un ejemplo utilizando etiquetas semánticas:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>HTML5</title>
  </head>
```



```
<body>
  <header>
    <nav></nav>
  </header>
  <main>
    <section>
      <article></article>
    </section>
    <aside></aside>
  </main>
  <footer></footer>
</body>
</html>
```

La siguiente imagen muestra una disposición posible para estos elementos. Pero ojo, porque para obtenerla, hay que usar CSS.



## Elemento `<header>` #

El elemento `<header>` contiene la cabecera con contenido introductorio para la sección de la página en que aparece.

Representa un grupo de ayudas introductorias o de navegación. Puede contener algunos elementos de encabezado, así como también un logo, un formulario de búsqueda, un nombre de autor y otros componentes.

Es habitual que contenga los elementos de encabezado (`<h1>` ... `<h6>`).

## Elemento `<aside>` #

El elemento `<aside>` se utiliza para contenido parcialmente relacionado con el contenido principal.

Estas secciones son a menudo representadas como **barras laterales** o como inserciones y contienen una explicación al margen como una definición de glosario, elementos relacionados indirectamente, como publicidad, la biografía del autor, o en aplicaciones web, la información de perfil o enlaces a blogs relacionados.

**No tiene por qué mostrarse en un lateral.**

## Elemento `<footer>` #

El elemento `<footer>` representa un pie de página para el contenido de sección más cercano o el elemento raíz de sección.

Un pie de página típicamente contiene información acerca del autor de la sección, datos de derechos de autor o enlaces a documentos relacionados.

**No tiene por qué mostrarse al final.**

## Elemento `<section>` #

El elemento de HTML `<section>` representa una sección genérica de un documento. Sirve para determinar qué contenido corresponde a qué parte de un esquema.

Pensemos en el esquema como en el índice de contenido de un libro: un tema común y subsecciones relacionadas. No se debe usar el elemento `<section>` como un mero contenedor genérico. Para esto, ya existe `<div>`, especialmente si el objetivo solamente es aplicar un estilo CSS a la sección.

## Elemento `<article>` #

El Elemento article de HTML `<article>` representa una composición autocontenida en un documento, página, una aplicación o en el sitio, que se destina a distribuir de forma independiente o reutilizable, por ejemplo, en la indicación.

Podría ser un mensaje en un foro, un artículo de una revista o un periódico, una entrada de blog, un comentario de un usuario, un widget interactivo o gadget, o cualquier otro elemento independiente del contenido.

## Elemento `<nav>` #

El elemento `<nav>` representa una sección de una página cuyo propósito es proporcionar enlaces de navegación, ya sea dentro del documento actual o a otros documentos.

Ejemplos comunes de secciones de navegación son: menús, tablas de contenido e índices.

## Elementos tipo texto #

### Encabezados y párrafos #

Para agrupar el texto en párrafos se usa el elemento `<p>`. Es un elemento de bloque (*block*).

```
<p>Texto del párrafo</p>
```



Para los encabezados, en HTML se definen 6 elementos (6 niveles):

```
<h1>Encabezado de nivel 1</h1>
<h2>Encabezado de nivel 2</h2>
<h3>Encabezado de nivel 3</h3>
<h4>Encabezado de nivel 4</h4>
<h5>Encabezado de nivel 5</h5>
<h6>Encabezado de nivel 6</h6>
```



Cuanto menor es el número, mayor es la importancia del encabezado. Es decir, el `<h1>` es el de mayor relevancia. El texto marcado debe servir como encabezado a la sección en la que aparece.

Los encabezados se pueden utilizar para organizar jerárquicamente el contenido de la página.

Un ejemplo en el cual se muestra el uso de los encabezados y párrafos:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Párrafos y encabezados</title>
  </head>
  <body>
    <h1>Ciclos formativos de la familia de IFC</h1>
```



```
<h2>Ciclos medios (CM)</h2>

<h3>SMR</h3>
<p>Sistemas Microinformáticos y Redes es un ciclo formativo de grado medio.</p>
<p>Incluye módulos como: Hardware, Software, Redes, etc.</p>

<h2>Ciclos superiores (CS)</h2>

<h3>ASIR</h3>
<p>Administración de Sistemas Informáticos en Red es un ciclo formativo de gr

<h3>DAM</h3>
<p>Desarrollo de Aplicaciones Multiplataforma es un ciclo formativo de grado

<h3>DAW</h3>
<p>Desarrollo de Aplicaciones Web es un ciclo formativo de grado superior.</p>
</body>
</html>
```

El navegador lo muestra de la siguiente manera:

# Ciclos formativos de la familia de IFC

## Ciclos medios (CM)

### SMR

Sistemas Microinformáticos y Redes es un ciclo formativo de grado medio.

Incluye módulos como: Hardware, Software, Redes, etc.

## Ciclos superiores (CS)

### ASIR

Administración de Sistemas Informáticos en Red es un ciclo formativo de grado superior.

### DAM

Desarrollo de Aplicaciones Multiplataforma es un ciclo formativo de grado superior.

### DAW

Desarrollo de Aplicaciones Web es un ciclo formativo de grado superior.

Pruébalo en el navegador pulsando aquí

## Saltos de línea y espacios en blanco #

Los navegadores, al procesar código HTML, **ignoran los saltos de línea** y, si encuentran varios espacios consecutivos, los reducen a uno solo (colapsan los espacios).

Si necesitas introducir un salto de línea, utiliza el elemento `<br>`:

```
Esto es una línea<br>
Esto es otra línea<br/>
```



Para introducir múltiples espacios, puedes usar la entidad HTML `&nbsp;`;

<p>Uno&ampnbsp&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&dos</p>



**Nota:** También puedes usar etiquetas HTML adicionales o propiedades CSS para controlar los espacios y saltos de línea.

## Ejemplo: Espacios y saltos de línea #

[illegible]

## Semántica a nivel de texto #

En este apartado se introducen algunos elementos HTML útiles a nivel semántico.

## Elemento `<a>` #

Convierte el texto encerrado entre las etiquetas en un hipervínculo:

```
<a href="https://es.wikipedia.org/">Wikipedia</a>
```



El atributo más importante es `href`, que indica la URL del vínculo. A veces se usa `href="#"` para referirse a la propia página.

El atributo `target` permite elegir dónde se abrirá el vínculo. Los valores más usados son:

| Valor                      | Descripción                                          |
|----------------------------|------------------------------------------------------|
| <code>target=_blank</code> | El enlace se abre en una ventana/pestaña nueva.      |
| <code>target=_self</code>  | El enlace se abre en la misma ventana (por defecto). |

Ejemplo:

```
<a href="https://es.wikipedia.org/" target="_blank">Wikipedia</a>  
<a href="https://es.wikipedia.org/" target="_self">Wikipedia</a>
```



## Elemento `<strong>` #

Representa que el texto marcado es importante:

```
El texto es <strong>importante</strong>
```



## Elemento `<em>` #

Indica énfasis (texto en cursiva):

```
Un elemento HTML puede ser <em>inline</em>
```



## Elemento `<br>` #



Introduce un salto de línea:

```
Línea 1<br>
Línea 2<br>
Línea 3
```



## Elemento `<small>` #

Permite insertar comentarios accesorios o texto en letra pequeña:

```
Texto marcado <small>con el elemento small</small>
```



## Elemento `<abbr>` #

Indica el significado de una abreviatura o sigla:

```
<abbr title="Hypertext Markup Language">HTML</abbr>
```



## Ejemplo completo #

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Semántica a nivel de texto</title>
  </head>
  <body>
    Texto marcado como <strong>importante</strong>.<br>
    Texto con <em>énfasis</em><br>
    Texto marcado <small>con el elemento small</small><br>
    Pulsa <a href="https://www.w3.org/">aquí</a> para ir a la página del <ab
  </body>
</html>
```



El resultado en el navegador sería:

Texto marcado como **importante**.

Texto con *énfasis*

Texto marcado con el elemento small

Pulsa aquí para ir a la página del W3C.

Compruébalo en el navegador pulsando aquí



#### CONSULTA:

En la web [MDN Web Docs](#) puedes encontrar más información sobre los elementos HTML.

## Listas #

Las listas en HTML se clasifican en tres tipos principales:

- **Listas ordenadas** (`<ol>`): Los elementos se numeran automáticamente.
- **Listas desordenadas** (`<ul>`): Los elementos se marcan con viñetas.
- **Listas de definición** (`<dl>`): Se usan para pares término-definición.

### Listas ordenadas #

```
<ol>
  <li>Elemento 1 de una lista ordenada</li>
  <li>Elemento 2 de una lista ordenada</li>
  <li>Elemento 3 de una lista ordenada</li>
</ol>
```



### Listas desordenadas #

```
<ul>
  <li>Elemento de una lista desordenada</li>
  <li>Elemento de una lista desordenada</li>
</ul>
```



```
<li>Elemento de una lista desordenada</li>
</ul>
```

## Listas de definición #

```
<dl>
  <dt>Coche</dt>
  <dd>Automóvil destinado al transporte de personas y con capacidad no superior
  <dt>Ordenador</dt>
  <dd>Máquina electrónica que, mediante determinados programas, permite almacena
  <dt>Bolígrafo</dt>
  <dd>Instrumento para escribir que tiene en su interior un tubo de tinta especí
</dl>
```



### EJEMPLO VISUAL

Puedes ver un ejemplo completo de listas en HTML [aquí](#).

## Tablas #

Las tablas sirven para mostrar datos estructurados en filas y columnas.



### USO DE LAS TABLAS

No se recomienda usar tablas para maquetar el diseño de una web. Utilízalas solo para datos tabulares.

## Elementos principales de una tabla #

Los elementos para definir una tabla son los siguientes (no es necesario usar todos):

| Elemento                   | Descripción                       |
|----------------------------|-----------------------------------|
| <code>&lt;table&gt;</code> | Delimita el contenido de la tabla |
| <code>&lt;tr&gt;</code>    | Define una fila                   |
| <code>&lt;td&gt;</code>    | Celda de datos                    |

|                               |                               |
|-------------------------------|-------------------------------|
| <code>&lt;th&gt;</code>       | Celda de cabecera             |
| <code>&lt;caption&gt;</code>  | Leyenda o título de la tabla  |
| <code>&lt;thead&gt;</code>    | Agrupar la cabecera           |
| <code>&lt;tbody&gt;</code>    | Agrupar el cuerpo de la tabla |
| <code>&lt;tfoot&gt;</code>    | Agrupar el pie de la tabla    |
| <code>&lt;colgroup&gt;</code> | Agrupar columnas              |

## Ejemplo básico de tabla #

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Tablas</title>
  </head>
  <body>
    <table>
      <caption>Tabla de socios</caption>
      <tr>
        <th>Nombre</th>
        <th>Apellido</th>
        <th>Edad</th>
      </tr>
      <tr>
        <td>Juan</td>
        <td>Puertas</td>
        <td>54</td>
      </tr>
      <tr>
        <td>Eva</td>
        <td>Montes</td>
        <td>44</td>
      </tr>
    </table>
  </body>
</html>
```



## ❗ PROBAR EN EL NAVEGADOR

Puedes ver el resultado de esta tabla en HTML [aquí](#).

**Nota:** Por defecto, las tablas no tienen bordes. Para añadirlos, utiliza CSS.

## Formularios #

Los formularios permiten recoger información que el usuario introduce en el navegador para un posterior tratamiento.

Es importante **validar los datos** introducidos para detectar los posibles errores cometidos por un usuario. La validación consiste en revisar que los datos introducidos se ajustan a unos requisitos. Por ejemplo, si se solicita un correo, que éste tenga la estructura de un correo. Otro ejemplo sería, que si se solicita el nombre, que no contenga números o caracteres especiales.

La **validación**, preferiblemente, se debe realizar en local, esto es, en el propio equipo, sin enviar los datos al servidor. De este modo, se evita sobrecargar la red con datos erróneos y sobrecargar al servidor con tareas innecesarias.

Normalmente se combinan los formularios con código JavaScript, lenguaje que ayuda a realizar esas validaciones, aunque HTML5 incluye algunos atributos que permiten realizar algunos tipos de validaciones sin programar código.

## Ejemplo completo de formulario #

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Ejemplo de Formulario</title>
  </head>
  <body>
    <h2>Bienvenido al registro de usuarios de nuestro foro</h2>
    <form action="">
      <fieldset>
        <legend>Introduzca su nombre de usuario y contraseña</legend>
        <div>
          <label>Usuario</label>
          <input required />
        </div>
        <br />
      </fieldset>
    </form>
  </body>
</html>
```



```

    <div>
      <label>Contraseña</label>
      <input type="password" required />
      <label>Repita Contraseña</label>
      <input type="password" required />
    </div>
  </fieldset>
  <br /><br />
  <label>Describa a continuación sus intereses:</label>
  <br />
  <textarea name="area" cols="60" rows="6"></textarea>
  <br /><br />
  Por último, seleccione el sistema operativo de su ordenador:
  <select name="Sistema operativo favorito">
    <option value="Linux">Linux </option>
    <option value="Windows">Windows </option>
    <option value="MacOS">Mac OS </option>
  </select>
  <br /><br />
  <div>
    <input type="checkbox" name="conforme" checked />
    Estoy conforme con la política de privacidad de la página
  </div>
  <div>
    <button onclick="alert('Hola mundo!');">Registrarme</button>
  </div>
</form>
</body>
</html>

```

En el navegador se vería así:

## Bienvenido al registro de usuarios de nuestro foro

Introduzca su nombre de usuario y contraseña

Usuario

Contraseña  Repita Contraseña

Describa a continuación sus intereses:

Por último, seleccione el sistema operativo de su ordenador: Linux ▼

☒ Estoy conforme con la política de privacidad de la página



### EJEMPLO VISUAL

Puedes ver este formulario funcionando [aquí](#).

## Declaración de un formulario #

La apertura y cierre de un formulario se hace mediante el elemento `<form>`:

```
<form name="mi-formulario" action="accion.php" method="GET"></form>
```



## Atributos de `<form>` #

LA etiqueta permite especificar una serie de atributos para ajustar sus características:

| Atributo | Descripción                                             |
|----------|---------------------------------------------------------|
| name     | Nombre del formulario.                                  |
| action   | Acción que se ejecuta al enviar el formulario.          |
| enctype  | Formato en el que se envían los valores del formulario. |

method

Método de envío HTTP: `GET` o `POST`.

## Métodos GET y POST #

Método `GET`:

- Permite pasar valores en ASCII con un límite de 100 caracteres.
- Los valores de las variables que se transmiten se pueden ver en la URL. Van a continuación de un `?` y, si existen múltiples valores, se separan por el símbolo de `&`.
- Ejemplo de URL con método GET:

```
https://web.com/index.php?variable1=valor1&variable2=valor2
```



Método `POST`:

- Permite pasar valores de variables y otros elementos como archivos.
- Carece de restricciones de longitud como el método `GET`.
- Las variables y sus valores no son visibles en la URL.

## Campos de formulario #

Dentro de un formulario puede haber varios tipos de controles:

- Campos de texto
- Contraseñas
- Selector de fecha
- Botones de opción múltiple (radio)
- Casillas de verificación (checkbox)

## Atributo name #

Un formulario está pensado para recoger información de forma estructurada y ser enviada a un servidor para su tratamiento. Para conseguirlo, es necesario que el servidor pueda identificar todos los campos de un formulario. Esto se consigue con el atributo `name`.

El valor que indiquemos en el atributo `name` de un campo de formulario, será el identificador que se deberá usar para obtener la información introducida por el usuario.



Por ejemplo, supongamos un formulario como el siguiente:

El código HTML correspondiente al formulario anterior sería el siguiente:

```
<form action="" method="GET">
  <input type="text" name="direccion" />
  <input type="submit" value="Enviar" />
</form>
```



En el campo anterior se ha indicado direccion como valor del atributo name. Cuando el usuario envíe ese formulario al servidor, éste deberá utilizar el identificador direccion para obtener el contenido introducido por el usuario en ese campo.

Por ejemplo, supongamos que el usuario completa el formulario con el siguiente contenido y que pulsa Enviar:

Al utilizar el método GET, la información se envía a través de la URL de la siguiente forma:

```
https://8mexj2.csb.app/?direccion=Santiago
```



De esta URL, todo lo que está a la derecha de ? es información en forma de parámetros. Es decir, lo siguiente:

```
?direccion=Santiago
```



Vemos que direccion tiene como valor Santiago, que es lo que ha introducido el usuario. El servidor recibe esa información a través de la URL de la petición HTTP y podrá manipularla.

Si utilizamos el método POST, la información se envía a través del payload de la petición HTTP, la cual tiene menos limitaciones.

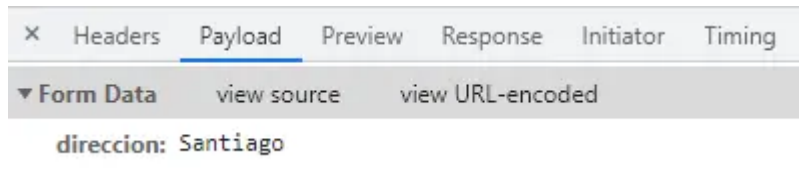
```
<form action="" method="POST">
  <input type="text" name="direccion" />
```



```
<input type="submit" value="Enviar" />
</form>
```

Este método, por ejemplo, es el único que podemos utilizar para enviar imágenes al servidor.

Utilizando este método, la información se podrá ver únicamente con herramientas de desarrolladores, las cuales nos permitirían ver la información enviada.



Resumiendo, independientemente del método usado (`GET` o `POST`), todos los campos de un formulario deben llevar el atributo `name` para que puedan ser utilizados por un servidor. Aunque es interesante, no es necesario entender como funcionan las peticiones HTTP para la creación de formularios.

## Campo de texto #

Para crear un campo de texto donde un usuario puede introducir datos, debemos utilizar el elemento `<input>`:

```
<input type="text" />
```



El elemento `<input>` se puede usar para varios tipos de control. Este elemento **no tiene etiqueta de cierre**.

El tipo de control viene dado por el valor del atributo **type**. Algunos ejemplos son:

```
<input type="text" />
<input type="radio" />
<input type="checkbox" />
```



El primero muestra un campo de texto normal (valor por defecto) y, el segundo, muestra un botón de radio. El tercero, una casilla de verificación.

## Atributos de `<input>` #

Además de `type`, el elemento `<input>` puede tener otros atributos que permiten definir su comportamiento:

| Atributo    | Descripción                                              |
|-------------|----------------------------------------------------------|
| name        | Nombre del campo.                                        |
| size        | Número de caracteres visibles. Por defecto, 20.          |
| maxlength   | Número máximo de caracteres permitidos.                  |
| value       | Valor por defecto.                                       |
| placeholder | Texto sugerido (gris, desaparece al enfocar).            |
| readonly    | El valor no puede ser modificado.                        |
| autofocus   | El cursor se sitúa en el campo al cargar la página.      |
| required    | El formulario no se puede enviar si el campo está vacío. |

Ejemplo:

```
<input type="text" name="usuario" size="30" maxlength="20" placeholder="Nombre de usuario" />
```

## Área de texto #

Permite recoger información abierta del usuario pero permitiendo un número mayor de caracteres. Se define con el elemento `<textarea>`:

```
<textarea name="area" rows="6" cols="60"></textarea>
```

| Atributo | Descripción       |
|----------|-------------------|
| name     | Nombre del campo. |
| rows     | Número de filas.  |

|             |                                                     |
|-------------|-----------------------------------------------------|
| cols        | Número de columnas.                                 |
| placeholder | Texto sugerido.                                     |
| maxlength   | Número máximo de caracteres.                        |
| required    | Campo obligatorio.                                  |
| readonly    | El valor no puede ser modificado.                   |
| autofocus   | El cursor se sitúa en el campo al cargar la página. |

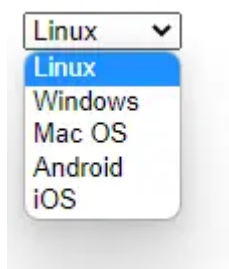
## Lista desplegable #

El elemento `<select>` permite crear una lista de opciones:

```
<select name="os">
  <option value="linux">Linux</option>
  <option value="windows">Windows</option>
  <option value="macos">Mac OS</option>
  <option value="android">Android</option>
  <option value="ios">iOS</option>
</select>
```



En el navegador se mostraría de la siguiente manera:



A continuación se muestran los atributos más importantes de `<select>` y `<option>`.

| Atributo | Descripción       |
|----------|-------------------|
| name     | Nombre del campo. |

|          |                                      |
|----------|--------------------------------------|
| size     | Número de opciones visibles.         |
| multiple | Permite seleccionar varias opciones. |

Cada `<option>` puede tener:

| Atributo | Descripción                      |
|----------|----------------------------------|
| value    | Valor enviado al servidor.       |
| selected | Opción seleccionada por defecto. |

## Botones de formulario #

Permite enviar el formulario a un destinatario. El destinatario depende del valor del atributo `action` de la etiqueta `<form>` que contiene el botón.

El destinatario puede ser una página en un servidor web destinada a procesar datos o puede ser una dirección de correo electrónico.

### Botón de envío:

```
<input type="submit" value="Enviar formulario" />
```



### Botón de anulación:

```
<input type="reset" value="Vaciar formulario" />
```



El atributo `value` indica el texto que aparecerá en el botón.

## Otros campos #

Dependiendo de los valores que queramos recoger en el formulario, podemos usar otros tipos de campos, más apropiados al tipo de dato.

**Checkbox:** permite que el usuario seleccione una o varias opciones.

| Atributo | Descripción                                    |
|----------|------------------------------------------------|
| name     | Nombre del campo.                              |
| value    | Valor enviado al servidor si está marcado.     |
| checked  | Indica si la casilla está marcada por defecto. |

```
<input type="checkbox" name="conforme" checked />
Acepto la política de privacidad
```



**Oculto:** permite enviar información que no es visible para el usuario.

```
<input type="hidden" />
```



**Contraseña:** permite ocultar la entrada del usuario (aparecerán asteriscos o puntos en lugar de los caracteres reales). Útil para recoger contraseñas.

```
<input type="password" />
```



**Ficheros:** Permite seleccionar un fichero del sistema de archivos del usuario.

```
<input type="file" />
```



### PARA ENVIAR FICHEROS

El formulario debe tener `method="POST"` y `enctype="multipart/form-data"`.

**Correo electrónico:** valida automáticamente que el formato del correo es correcto.

```
<input type="email" />
```



**URL:** valida automáticamente que el formato de la URL es correcto.



```
<input type="url" />
```

**Números enteros:** permite seleccionar un número entero dentro de un rango.

```
<input type="number" min="1" max="10" step="2" />
```



| Atributo | Descripción   |
|----------|---------------|
| min      | Valor mínimo. |
| max      | Valor máximo. |
| step     | Incremento.   |

## Fechas #

Dependiendo del tipo de fecha que queramos recoger, podemos utilizar diferentes variaciones en las etiquetas:

```
<input type="datetime-local" /> <!-- Día, mes, año y hora -->
<input type="date" /> <!-- Día, mes y año -->
<input type="month" /> <!-- Mes y año -->
<input type="week" /> <!-- Semana -->
<input type="time" /> <!-- Hora -->
```



Con cada uno de estos tipos, el navegador mostrará un selector adecuado para recoger la fecha o la hora (o ambas).

## Rangos #

Permite seleccionar un valor dentro de un rango, con un control deslizante:

```
<input type="range" min="0" max="100" step="5" value="50" />
```



| Atributo | Descripción |
|----------|-------------|
|----------|-------------|

|       |                |
|-------|----------------|
| min   | Valor mínimo.  |
| max   | Valor máximo.  |
| step  | Incremento.    |
| value | Valor inicial. |

## Organización de formularios #

Para la organización de los campos de un formulario disponemos de varios elementos. A continuación, se muestran algunos.

### Etiqueta `<label>`:

El elemento `<label>` permite asociar a cada campo del formulario una etiqueta con su nombre. El texto mostrado entre las etiquetas `<label>` se muestra y constituye, además, una ayuda de **usabilidad a personas invidentes**. El elemento `<label>` puede asociarse a un campo mediante el atributo `for`, cuyo valor debe coincidir con el atributo `id` del campo asociado. Veamos un ejemplo:

```
<label for="conforme">Acepto el acuerdo de licencia</label>
<input type="checkbox" name="licencia" id="conforme" value="ok" />
```



### Elemento `<fieldset>` y `<legend>`:

El elemento `<fieldset>` permite agrupar varios campos relacionados dentro de un formulario. El elemento `<legend>` permite añadir una leyenda o título al grupo de campos.

```
<fieldset>
  <legend>Datos de acceso</legend>
  <label for="username">Usuario</label>
  <input type="text" id="username" name="user" required />
  <br />
  <label for="password">Contraseña</label>
  <input type="password" id="password" name="pass" required />
</fieldset>
```



## Validación de formularios #



Durante la creación de formularios web, se vuelve necesario realizar una verificación donde se compruebe que los todos los campos se envían correctamente al servidor.

Para ver cómo se realiza, consideremos el siguiente formulario:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Formulario de registro</title>
  </head>
  <body>
    <h1>Formulario de registro</h1>

    <form action="registro.php" method="get">
      <p>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre" maxlength="50" />
      </p>

      <p>
        <label for="ciclo">Ciclo formativo</label>
        <input type="radio" id="dam" name="ciclo" value="dam" />
        <label for="dam">DAM</label>
        <input type="radio" id="daw" name="ciclo" value="daw" />
        <label for="daw">DAW</label>
        <input type="radio" id="asir" name="ciclo" value="asir" />
        <label for="asir">ASIR</label>
      </p>

      <p>
        <input type="checkbox" id="info" name="info" checked="checked" />
        <label for="info">Deseo recibir información sobre novedades.</label>
      </p>

      <p>
        <input type="submit" value="Enviar" />
      </p>
    </form>
  </body>
</html>
```

Para realizar la comprobación, vamos a usar una funcionalidad que nos proporciona las **herramientas de desarrollador** (developer tools) del navegador. En este apartado vamos a ver cómo se realiza en

Google Chrome, Firefox o Microsoft Edge.

A continuación, se describen los pasos.

## Paso 1. Abrir las herramientas de desarrollador #

Una vez abierto el HTML del formulario:

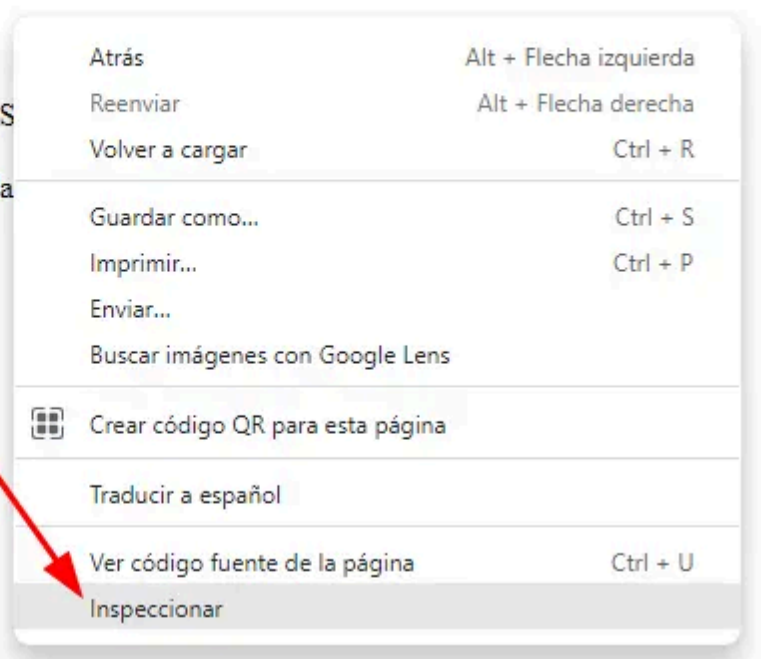
- Botón derecho sobre cualquier parte de la página web
- Pulsamos en Inspeccionar (Inspect).

## Formulario de registro

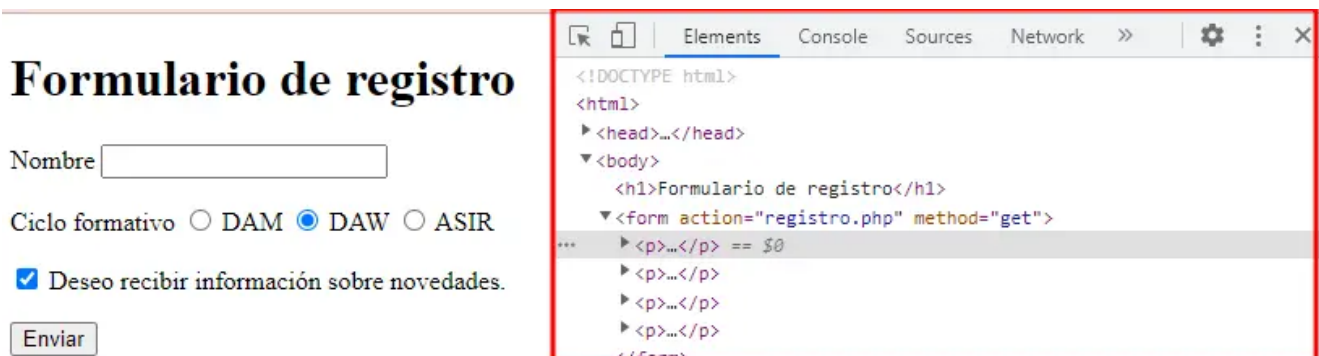
Nombre

Ciclo formativo ☐ DAM ☒ DAW ☐ ASIR

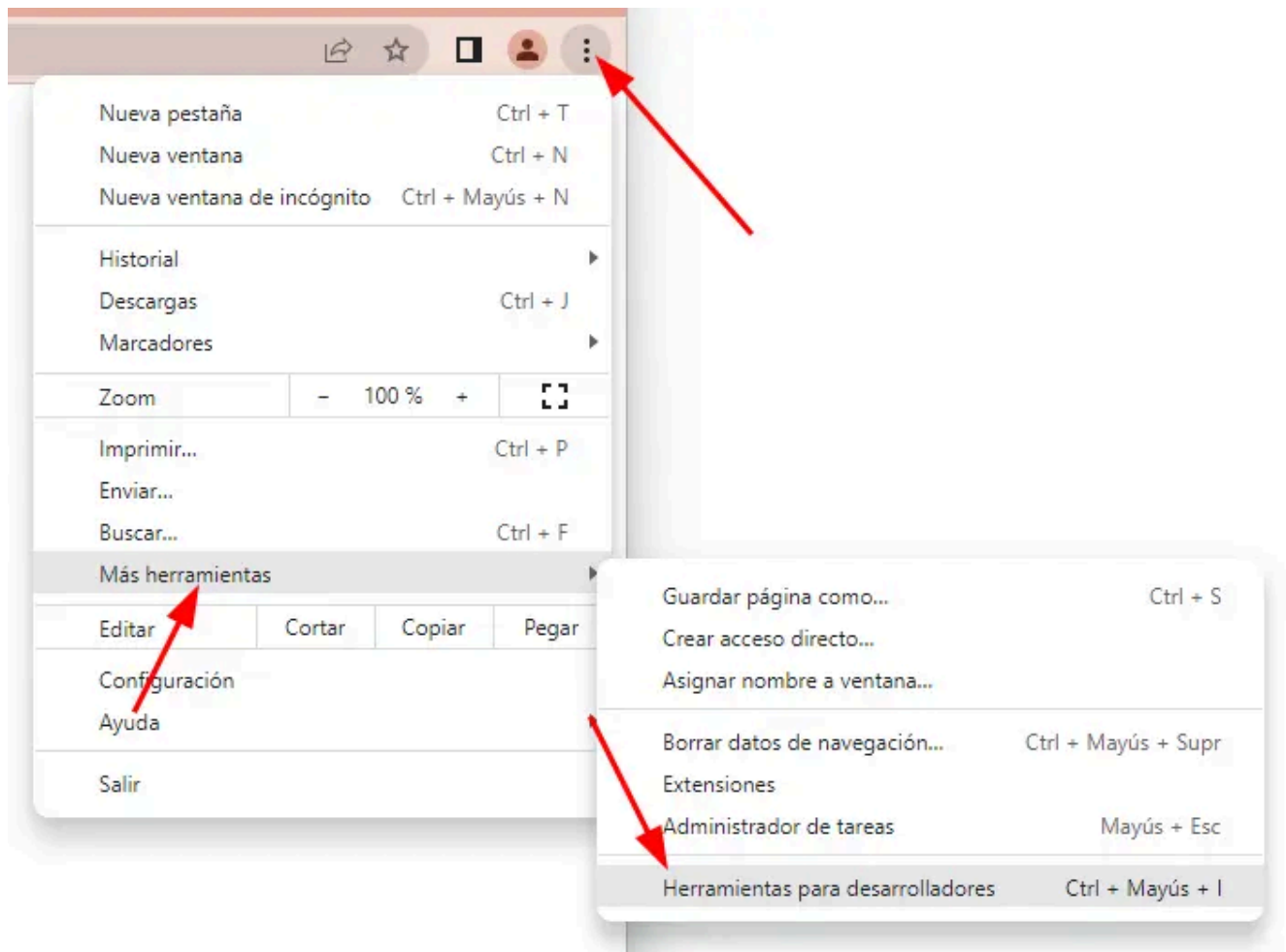
☒ Deseo recibir información sobre novedades.



Esta acción abrirá un menú como el siguiente:

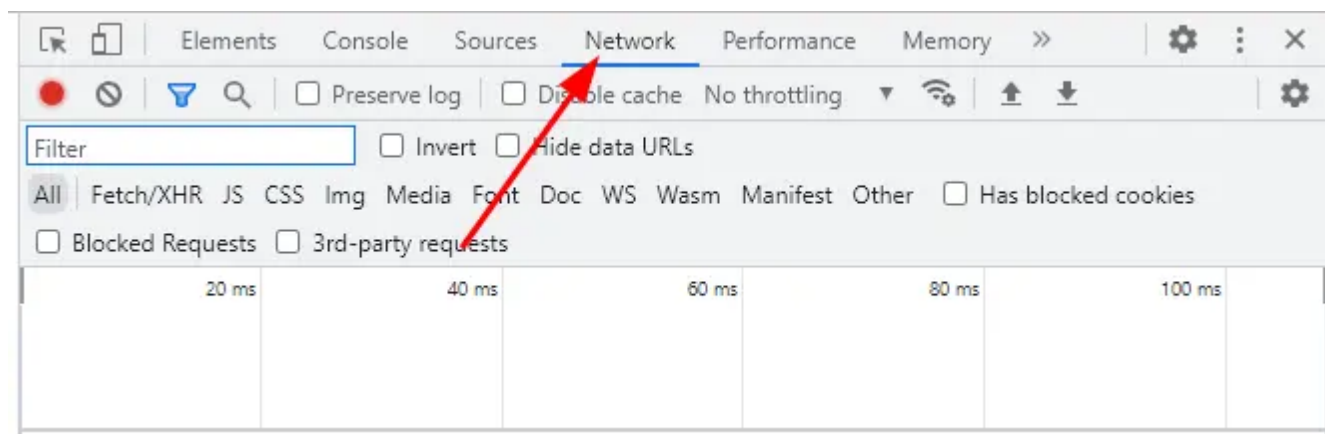


De forma alternativa, se pueden abrir las herramientas de desarrollador de la siguiente manera:



## Paso 2. Pestaña *Red* #

A continuación, clicamos en la opción Red (Network), lo que abre un panel como el siguiente:

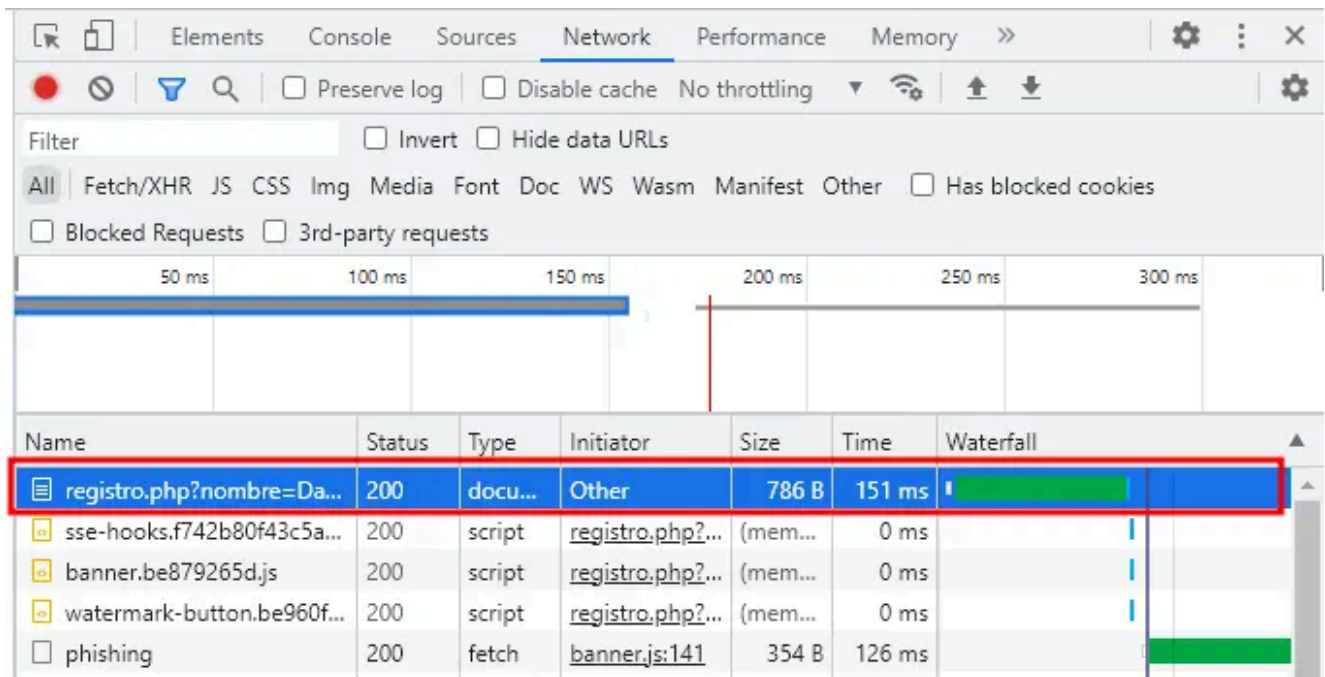


Ese panel muestra todas las peticiones HTTP que intercambia el navegador con el servidor. Cada petición HTTP se corresponde con una fila dentro de ese panel.

## Paso 3. Enviar formulario #

Una vez tenemos ese panel abierto, cubrimos los datos del formulario y le damos al botón de Enviar.

Al clicar en el botón Enviar, el formulario va a enviar los datos al servidor. El destino de los datos es el indicado en el atributo action de `<form>`. En este ejemplo, registro.php. Esto generará una nueva entrada en la lista de eventos del panel Network (aunque puede no ser la única).



En la imagen anterior se resalta la petición correspondiente con el envío de datos del formulario. Vemos que empieza por registro.php, que es la ruta indicada en el atributo action de `<form>`.

#### Paso 4. Accedemos a los detalles de la petición #

Pulsamos sobre la fila, lo que permite abrir un panel donde aparece toda la información relacionada con la petición HTTP. Este panel tiene el siguiente aspecto:

The screenshot shows the Network tab of a web browser's developer tools. A request to `registro.php?nombre=David...` is selected. The 'Headers' sub-tab is active, showing the following details:

- General:**
  - Request URL: `https://jkr7ng.csb.app/registro.php?nombre=David&ciclo=dam&info=on`
  - Request Method: GET
  - Status Code: 200
  - Remote Address: 172.64.151.11:443
  - Referrer Policy: strict-origin-when-cross-origin
- Response Headers:**
  - `alt-svc: h3=":443"; ma=86400, h3-29=":443"; ma=86400`
  - `cache-control: private, max-age=0, no-cache, no-store`
  - `cf-cache-status: DYNAMIC`
  - `cf-ray: 75c8cb0d8efc866c-MAD`
  - `content-encoding: br`
  - `content-type: text/html`
  - `date: Wed, 19 Oct 2022 10:22:32 GMT`
  - `server: cloudflare`
  - `set-cookie: signedIn=; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; max-age=0; HttpOnly`
  - `vary: Accept-Encoding`
  - `via: 1.1 google`
  - `x-request-id: Fx9xSquIc85Zx7wPaaKE`
- Request Headers:**
  - `:authority: jkr7ng.csb.app`
  - `:method: GET`
  - `:path: /registro.php?nombre=David&ciclo=dam&info=on`
  - `:scheme: https`
  - `accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9`
  - `accept-encoding: gzip, deflate, br`
  - `accept-language: es-ES,es;q=0.9`

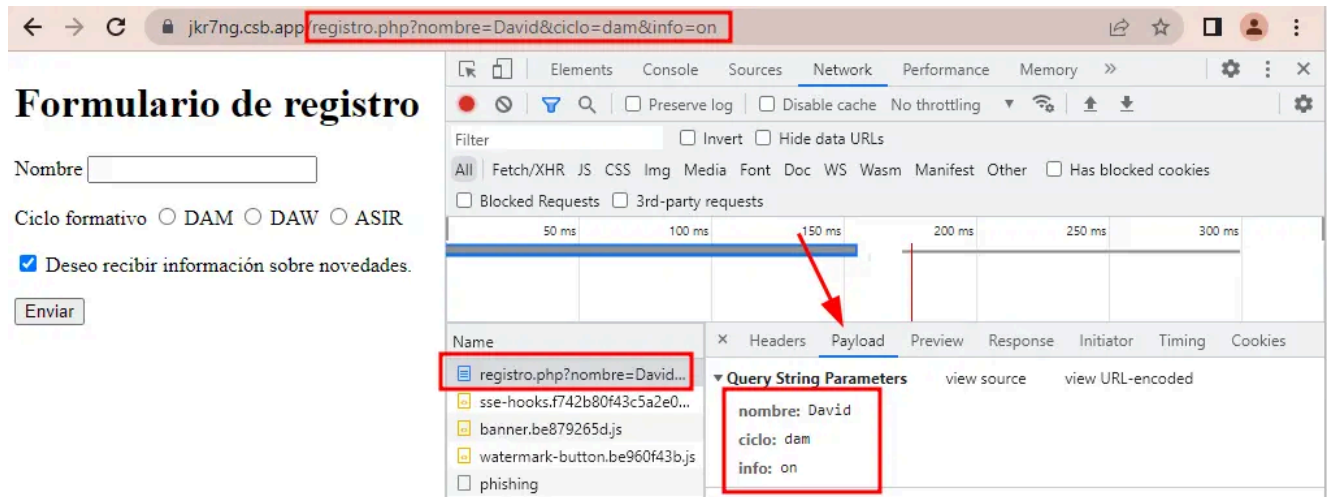
At the bottom, it indicates '5 requests' and '1.1 kB transferred'.

De las pestañas que ahí aparecen, para esta explicación nos interesan dos:

- **Headers** (encabezados): contiene información relacionada con la cabecera del mensaje. En la captura anterior podemos ver el campo `Request Method` indica que se ha enviado la petición mediante el método `GET`. Si se hubiera usado el método `POST` en ese campo aparecería `POST`.
- **Payload** (carga útil): contiene la información enviada por el formulario al servidor.

## Paso 5. Pestaña Payload #

Si pulsamos en Payload, podemos ver qué datos se han enviado. Al ser un formulario enviado mediante el método GET, podemos también visualizar los datos en la URL.



En la imagen anterior se puede visualizar como se han enviado los campos nombre, ciclo e info.

Cuando se cubre el formulario y se pulsa en el botón de *Enviar*, se genera un fichero formado por una **estructura de datos clave-valor**, donde:

- La **clave** es el valor del atributo name de cada elemento
- El **valor** es el texto introducido en los campos input o el valor del atributo del campo value de cada elemento.

De esta forma, podemos ver que al enviar el formulario:

- Para el campo `nombre` el valor es el texto introducido por el usuario en el input.
- Para el campo `ciclo` el valor es el asignado al atributo value del radio button seleccionado.

Si a la hora de enviar el formulario alguno de los campos no aparece en esa pestaña Payload, pueden ocurrir dos situaciones:

- No se ha escrito un atributo `name` en la etiqueta correspondiente.
- No se ha seleccionado el *checkbox* o *radio button* correspondiente.

Además, si el valor que aparece en el campo radio al enviar el formulario es el valor `on` significa que no se ha escrito el atributo value.

Con este ejemplo se puede observar la importancia de los campos `name` y `value` dentro de un formulario.

## Validación #

Es posible validar si nuestro código HTML cumple con la especificación de HTML5.

Para ello, existen diferentes páginas en Internet que nos facilitan este servicio:

- W3C Markup Validation Service